Introducción a la Programación

Guía Práctica 9
Testing de caja blanca



Departamento de Computación
Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires

Ejercicio 1. ★ Sea el siguiente programa:

```
def max (x: int, y: int) -> int:
L1:   result: int = 0
L2:   if x < y:
L3:      result = y
      else:
L4:      result = x
L5:   return result
```

Y los siguientes casos de test:

- test1:
  - Entrada x = 0, y=0
  - Resultado esperado result=0
- test2:
  - Entrada x = 0, y=1
  - Resultado esperado result=1

1. Describir el diagrama de control de flujo (control-flow graph) del programa `max`.
2. Detallar qué líneas del programa cubre cada test

Test	L1	L2	L3	L4	L5
test1					
test2					

3. Detallar qué decisiones (branches) del programa cubre cada test

Test	L2-True	L2-False
test1		
test2		

4. Decidir si la siguiente afirmación es verdadera o falsa: *"El test suite compuesto por test1 y test2 cubre el 100 % de las líneas del programa y el 100 % de las decisiones (branches) del programa"*. Justificar.

Ejercicio 2. ★ Sea la siguiente especificación del problema de retornar el mínimo elemento entre dos números enteros:

`problema min` (in x: $\mathbb{Z}$, in y: $\mathbb{Z}$) : $\mathbb{Z}$ {
`requiere:` $\{True\}$
`asegura:` $\{(x < y \rightarrow result = x) \wedge (x \geq y \rightarrow result = y)\}$
}

Un programador ha escrito el siguiente programa para implementar la especificación descripta:

```
def min (x: int, y: int) -> int:
L1:   result: int = 0
L2:   if x < y:
L3:      result = x
      else:
L4:      result = x
L5:   return result
```

Y el siguiente conjunto de casos de test (test suite):

- minA:
  - Entrada x=0,y=1
  - Salida esperada 0
- minB:
  - Entrada x=1,y=1
  - Salida esperada 1

1. Describir el diagrama de control de flujo (control-flow graph) del programa `min`.
2. ¿La ejecución del test suite resulta en la ejecución de todas las líneas del programa `min`?
3. ¿La ejecución del test suite resulta en la ejecución de todas las decisiones (branches) del programa?
4. ¿Es el test suite capaz de detectar el defecto de la implementación del problema de encontrar el mínimo?
5. Agregar nuevos casos de tests y/o modificar casos de tests existentes para que el test suite detecte el defecto.

Ejercicio 3. ★ Sea la siguiente especificación del problema de sumar y una posible implementación en lenguaje imperativo.

`problema sumar` (in x: $\mathbb{Z}$, in y: $\mathbb{Z}$) : $\mathbb{Z}$ {
`requiere:` $\{True\}$
`asegura:` $\{result = x + y\}$
}

```
def sumar (x: int, y: int) -> int:
L1:   result: int = 0
L2:   result = result + x
L3:   result = result + y
L4:   return result
```

1. Describir el diagrama de control de flujo (control-flow graph) del programa `sumar`.
2. Escribir un conjunto de casos de test (o "test suite") que ejecute todas las líneas del programa `sumar`.

Ejercicio 4. Sea la siguiente especificación del problema de restar y una posible implementación en lenguaje imperativo:

`problema restar` (in x: $\mathbb{Z}$, in y: $\mathbb{Z}$) : $\mathbb{Z}$ {
`requiere:` $\{True\}$
`asegura:` $\{result = x - y\}$
}

```
def restar (x: int, y: int) -> int:
L1:   result: int = 0
L2:   result = result + x
L3:   result = result + y
L4:   return result
```

1. Describir el diagrama de control de flujo (control-flow graph) del programa `restar`.
2. Escribir un conjunto de casos de test (o "test suite") que ejecute todas las líneas del programa `restar`.
3. La línea L3 del programa `restar` tiene un defecto, ¿es el test suite descripto en el punto anterior capaz de detectarlo? En caso contrario, modificar o agregar nuevos casos de test hasta lograr detectarlo.

Ejercicio 5. Sea la siguiente especificación del problema de `signo` y una posible implementación en lenguaje imperativo:

`problema signo` (in x: $\mathbb{R}$) : $\mathbb{Z}$ {
`requiere:` $\{True\}$
`asegura:` $\{(result = 0 \land x = 0) \lor (result = -1 \land x < 0) \lor (result = 1 \land x > 0)\}$
}

```
def signo(x: float) -> int:
L1:   result: int = 0
L2:   if x<0:
L3:      result = -1
L4:   elif x>0:
L5:      result = 1
L6:   return result
```

1. Describir el diagrama de control de flujo (control-flow graph) del programa `signo`.
2. Escribir un test suite que ejecute todas las líneas del programa `signo`.
3. ¿El test suite del punto anterior ejecuta todas las posibles decisiones ("branches") del programa?

Ejercicio 6. Sea la siguiente especificación del problema de `fabs` y una posible implementación en lenguaje imperativo:

`problema fabs` (in x: $\mathbb{R}$) : $\mathbb{R}$ {
    `requiere:` $\{True\}$
    `asegura:` $\{result = |x|\}$
}

```
def fabs(x: float) -> float:
L1:   result: float = 0
L2:   if x<0:
L3:      result = -x
L4:   return result
```

1. Describir el diagrama de control de flujo (control-flow graph) del programa `fabs`.
2. Escribir un test suite que ejecute todas las líneas del programa `fabs`.
3. Escribir un test suite que ejecute todas las posibles decisiones ("branches") del programa.
4. Escribir un test suite que ejecute todas las líneas del programa pero no ejecute todos las decisiones del programa.
5. ¿Los test suites de los puntos anteriores detectan el defecto en la implementación? De no ser así, modificarlos para que lo hagan.

Ejercicio 7. ★ Sea la siguiente especificación:

`problema fabs` (in x: $\mathbb{Z}$) : $\mathbb{Z}$ {
    `requiere:` $\{True\}$
    `asegura:` $\{result = |x|\}$
}

Y la siguiente implementación:

```
def fabs(x: int) -> int:
L1:   if x < 0:
L2:     return -x
      else:
L3:     return x
```

1. Describir el diagrama de control de flujo (control-flow graph) del programa `fabs`.
2. Escribir un test suite que ejecute todas las líneas y todos los branches del programa.

Ejercicio 8. ★ Sea la siguiente especificación del problema de `mult10` y una posible implementación en lenguaje imperativo:

`problema mult10` (in x: $\mathbb{Z}$) : $\mathbb{Z}$ {
    `requiere:` $\{True\}$
    `asegura:` $\{result = x * 10\}$
}

```
def mult10(x: int) -> int:
L1:  result: int = 0
L2:  count: int = 0
L3:  while(count < 10):
L4:     result = result + x
L5:     count = count + 1
L6:  return result
```

1. Describir el diagrama de control de flujo (control-flow graph) del programa `mult10`.
2. Escribir un test suite que ejecute todas las líneas del programa `mult10`.
3. ¿El test suite anterior ejecuta todas las posibles decisiones ("branches") del programa?

Ejercicio 9. Sea la siguiente especificación del problema de `sumar` y una posible implementación en lenguaje imperativo:

`problema sumar` (in x: $\mathbb{Z}$, in y: $\mathbb{Z}$) : $\mathbb{Z}$ {
    `requiere:` $\{True\}$
    `asegura:` $\{result = x + y\}$
}

```
def sumar(x: int , y: int) -> int:
L1:   sumando: int = 0
L2:   abs_y: int = 0
L3:   if y < 0:
L4:     sumando = -1
L5:     abs_y = -y
      else:
L6:     sumando = 1
L7:     abs_y = y
L8:   result: int = x
L9:   count: int = 0
L10:  while(count < abs_y):
L11:    result = result + sumando
L12:    count = count + 1
L13:  return result
```

1. Describir el diagrama de control de flujo (control-flow graph) del programa `sumar`.
2. Escribir un test suite que ejecute todas las líneas del programa `sumar`.
3. Escribir un test suite que ejecute todas las posibles decisiones ("branches") del programa.

Ejercicio 10. Sea el siguiente programa que computa el máximo común divisor entre dos enteros.

```
def mcd(x: int , y: int) -> int:
L1:  # requiere: x e y tienen que ser no negativos
L2:  tmp: int = 0
L3:  while(y != 0):
L4:    tmp = x % y
L5:    x = y
L6:    y = tmp
L7:  return x
```

1. Describir el diagrama de control de flujo (control-flow graph) del programa `mcd`.
2. Escribir un test suite que ejecute todas las líneas y todos los branches del programa.

Ejercicio 11. ★ Sea el siguiente programa que retorna diferentes valores dependiendo si a, b y c, definen lados de un triángulo inválido, equilátero, isósceles o escaleno.

```
        def triangle(a: int , b: int, c: int) -> int:
        L1:     if(a <= 0 or b <= 0 or c <= 0):
        L2:         return 4 # invalido
        L3:     if(not ((a + b > c) and (a + c > b) and (b + c > a))):
        L4:         return 4 # invalido
        L5:     if(a == b and b == c):
        L6:         return 1 # equilatero
        L7:     if(a == b or b == c or a == c):
        L8:         return 2 # isosceles
        L9:     return 3  # escaleno
```

1. Describir el diagrama de control de flujo (control-flow graph) del programa `triangle`.
2. Escribir un test suite que ejecute todas las líneas y todos los branches del programa.

Ejercicio 12. ★ Sea la siguiente especificación del problema de `multByAbs` y una posible implementación en lenguaje imperativo:

`problema multByAbs` (in x: $\mathbb{Z}$, in y:$\mathbb{Z}$, out result: $\mathbb{Z}$) {
`requiere:` $\{True\}$
`asegura:` $\{result = x * |y|\}$
}

```
def multByAbs(x: int, y: int) -> int:
L1:     abs_y: int = fabs(y); # ejercicio anterior
L2:     if abs_y < 0:
L3:        return -1
        else:
L4:        result: int = 0;
L5:        i: int = 0;
L6:        while i < abs_y:
L7:            result = result + x;
L8:            i += 1
L9:     return result
```

1. Describir el diagrama de control de flujo (control-flow graph) del programa `multByAbs`.
2. Detallar qué líneas y branches del programa no pueden ser cubiertos por ningún caso de test. ¿A qué se debe?
3. Escribir el test suite que cubra todas las líneas y branches que puedan ser cubiertos.

Ejercicio 13. Sea la siguiente especificación del problema de `vaciarSecuencia` y una posible implementación en lenguaje imperativo:

`problema vaciarSecuencia` (inout s: $seq\langle\mathbb{Z}\rangle$) {
`requiere:` {True}
`modifica:` {s}
`asegura:` $\{|s| = |s@pre| \land s[j] = 0$ para cualquier j tal que $0 \leq j < |s|\}$
}

```
          def vaciarSecuencia(s: [int]):
L1,L2,L3:     for i in range(len(s)):
L4:               s[i] = 0
```

1. Escribir el diagrama de control de flujo (control-flow graph) del programa `vaciarSecuencia`.
2. Escribir un test suite que cubra todos las líneas de programa (observar que un `for` contiene 3 líneas distintas).
3. En caso que el test suite del punto anterior no cubriera todo los branches del programa, extenderlo de modo que logre cubrirlos.

Ejercicio 14. Sea la siguiente especificación del problema de `existeElemento` y una posible implementación en lenguaje imperativo:

`problema existeElemento` (s: $seq\langle\mathbb{Z}\rangle$, e: $\mathbb{Z}$) : Bool {
`requiere:` $\{True\}$
`asegura:` $\{result = True \leftrightarrow s[j] = e$ para algún j tal que $0 \leq j < |s|\}$
}

```
          def existeElemento(s: [int], e: int) -> bool:
L1:           result: bool = False
L2,L3,L4:     for i in range(len(s)):
L5:               if s[i] == e:
L6:                   result = True
L7:                   break
L8:           return result
```

1. Escribir el diagrama de control de flujo (control-flow graph) del programa `existeElemento`.
2. Escribir un test suite que cubra todos las líneas de programa.
3. En caso que el test suite del punto anterior no cubriera todo los branches del programa, extenderlo de modo que logre cubrirlos.